

Каркас Пз1 — Ініціалізація проєкту

Сервер вашого майбутнього проєкту. Це **не готовий проєкт**: процес стартує, але порт не відкриває, і `/api/health` не існує. Довести це до робочого стану — і є Пз1.

Що всередині

Шлях	Стан	Роль
<code>server/package.json</code> , <code>server/tsconfig.json</code>	готове	версії зафіксовані до патча, ESM (<code>"type": "module"</code>)
<code>server/src/app.ts</code>	TODO(1)	фабрика застосунку; маршруту <code>/api/health</code> немає
<code>server/src/index.ts</code>	TODO(2), TODO(3)	точка входу; порт не відкривається
<code>server/src/types.ts</code>	готове + TODO(4)	тип <code>Health</code> дано; ваша головна сутність — ваша
<code>.gitignore</code>	готове	<code>node_modules</code> , <code>dist</code>
<code>README-шаблон.md</code>	шаблон	копіюєте в <code>README.md</code> і заповнюєте своїм проєктом

Про цей файл. Він описує каркас і у вашому репозиторії не потрібен. Ваш `README.md` — це `README-шаблон.md`: `cp README-шаблон.md README.md` (замінивши цей опис) і заповнити домен, сутності, стек і запуск. Заповнений `README` — один із пунктів того, що здається за Пз1.

Як запустити

```
cd server
npm install
npm run dev           # tsx watch; процес живий, але порт поки не слухає
```

Node — 22.12 або новіший. `package-lock.json` лежить у `server/`: версії зафіксовані до патча, тому у всій групі ставиться те саме. Лок-файл не видаляйте — комітьте разом із рештою.

Чотири TODO

Кожен має номер, адресу (`[пз1 · л1]`) і опис результату — не «допишіть тут».

#	Файл	Результат	Як видно, що не зроблено
1	server/src/app.ts	GET /api/health → 200 + JSON зі станом	будь-який шлях віддає 404
2	server/src/index.ts	процес слухає порт 3000	curl не отримує нічого, код 000
3	server/src/index.ts	невдалий старт дає зрозумілий лог і ненульовий код виходу	зайнятий порт виглядає як стектрейс
4	server/src/types.ts	тип вашої головної сутності (4+ поля)	у файлі лише health

TODO(1)–(3) закривають перевірку з розділу нижче. TODO(4) перевіряється очима на парі.

Як перевірити себе

Один `curl` — цього на Пз1 достатньо, окремого скрипта перевірки немає:

```
curl -i http://localhost:3000/api/health
```

На чистому каркасі `curl` не отримує нічого (`Connection refused` , код 000) — порт ніхто не відкрив. Коли з'явиться TODO(2), той самий `curl` почне повертати 404 — і це прогрес: сервер уже відповідає, просто маршруту ще немає. Здано, коли відповідь — `HTTP/1.1 200 OK` і тіло з `"status": "ok"` :

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status": "ok", "uptime": 12.4}
```

Сервер слухає інший порт — питайте його за тим адресою, яку він надрукував у лозі.

Чого тут немає

- **веб-частини.** Клієнт з'явиться в Пз5, після лекцій про React. Сьогодні інтерфейс вашого сервера — це HTTP і `curl` ;
- **автоматизованих скриптів і тестів.** Ні раннера, ні `npm test` , ні shell-скриптів: перевірка — один `curl` з цього README. Восьмиперевірочний контрактний скрипт з'явиться в Пз2;
- **.env і змінних середовища.** Порт поки константа 3000; коли в Пз3 і Пз7 з'являться справжні налаштування, тоді з'явиться і конфіг;

- **ESLint і Prettier.** До Пз7 це ваша особиста справа; у Пз7 лінт стає вимогою, і хто налаштує його сьогодні, той не виправляє сімсот попереджень за один вечір;
- **Зайвих залежностей.** `fastify`, `tsx`, `typescript`, `@types/node` — і все. Решту додаєте ви, і на захисті мусите вміти сказати, навіщо.